

Deep Galerkin Method (DGM): A Deep Learning Algorithm for Solving PDE

Implementation Report and Theoretical Analysis

Based on Sirignano & Spiliopoulos, *J. Comput. Phys.*, 375, 2018

Aryan Deo

April 15, 2026

Abstract

This report presents a comprehensive study of the Deep Galerkin Method (DGM), a meshfree deep learning algorithm for solving high-dimensional partial differential equations proposed by Sirignano and Spiliopoulos (2018). We provide the theoretical foundations including the neural network architecture, the loss function formulation, convergence guarantees, and a Monte Carlo scheme for efficient computation of second derivatives. We implement the DGM for four benchmark problems—the Black-Scholes PDE, the heat equation, Burgers’ equation, and the 2D Poisson equation—and present numerical results with error analysis.

Contents

1	Introduction	3
1.1	Contributions of the Original Paper	3
2	Mathematical Background	3
2.1	Problem Formulation	3
2.2	Neural Network Approximation	3
3	The DGM Algorithm	4
3.1	Stochastic Gradient Descent Formulation	4
3.2	DGM Network Architecture	4
3.3	Monte Carlo Method for Second Derivatives	5
4	Convergence Theory	6
4.1	Universal Approximation for PDEs	6
4.2	Convergence to the PDE Solution	6
5	Implementation and Numerical Results	6
5.1	Problem 1: Black-Scholes PDE	6
5.2	Problem 2: Heat Equation	7
5.3	Problem 3: Burgers’ Equation	7
5.4	Problem 4: 2D Poisson Equation	8
5.5	Summary of Results	9

6	Discussion	9
6.1	Advantages of DGM	9
6.2	Limitations	9
6.3	Comparison with PINNs	9
7	Conclusion	9

1 Introduction

High-dimensional partial differential equations (PDEs) arise in physics, engineering, and finance but pose severe computational challenges for traditional numerical methods. Finite difference methods require meshes of size $O(N^{d+1})$ where d is the spatial dimension, making them infeasible when d is even moderately large.

The Deep Galerkin Method (DGM) proposed by Sirignano and Spiliopoulos [1] addresses this challenge by approximating the PDE solution with a deep neural network trained via stochastic gradient descent on randomly sampled space-time points. The method is **meshfree**—no grid is ever constructed—which is the key to scaling to high dimensions.

The name “Deep Galerkin Method” reflects its analogy with classical Galerkin methods: instead of approximating the solution as a linear combination of basis functions, DGM uses a deep neural network as the approximator.

1.1 Contributions of the Original Paper

- (i) A meshfree deep learning algorithm for high-dimensional PDEs
- (ii) An LSTM-inspired neural network architecture (the DGM network)
- (iii) A Monte Carlo scheme for efficient computation of second derivatives
- (iv) Numerical results on free boundary PDEs in up to 200 dimensions
- (v) A convergence theorem for neural network approximations of quasilinear parabolic PDEs

2 Mathematical Background

2.1 Problem Formulation

Consider a general parabolic PDE with d spatial dimensions:

$$\frac{\partial u}{\partial t}(t, x) + \mathcal{L}u(t, x) = 0, \quad (t, x) \in [0, T] \times \Omega, \quad (1)$$

with initial and boundary conditions:

$$u(0, x) = u_0(x), \quad x \in \Omega, \quad (2)$$

$$u(t, x) = g(t, x), \quad x \in \partial\Omega, \quad (3)$$

where $\Omega \subset \mathbb{R}^d$, and $\mathcal{L}$ is a (possibly nonlinear) differential operator.

2.2 Neural Network Approximation

The DGM algorithm approximates $u(t, x)$ with a deep neural network $f(t, x; \theta)$ where $\theta \in \mathbb{R}^K$ are the network parameters. The key observation is that for any candidate function f , one can measure how well it satisfies the PDE *without knowing the true solution* by evaluating the residual.

Definition 1 (Objective Function). *The DGM objective function is:*

$$J(f) = \left\| \frac{\partial f}{\partial t} + \mathcal{L}f \right\|_{[0, T] \times \Omega, \nu_1}^2 + \|f - g\|_{[0, T] \times \partial\Omega, \nu_2}^2 + \|f(0, \cdot) - u_0\|_{\Omega, \nu_3}^2 \quad (4)$$

where $\|h(y)\|_{Y, \nu}^2 = \int_Y |h(y)|^2 \nu(y) dy$ and ν_i are positive probability densities.

If $J(f) = 0$, then f exactly solves the PDE. The goal is to find parameters θ minimizing $J(f(\cdot; \theta))$.

3 The DGM Algorithm

3.1 Stochastic Gradient Descent Formulation

Direct minimization of $J(f)$ is infeasible when d is large since it involves high-dimensional integrals. Instead, DGM uses stochastic gradient descent (SGD) on randomly sampled points.

Algorithm 1 Deep Galerkin Method (DGM)

- 1: **Initialize** neural network parameters θ_0
- 2: **repeat**
- 3: Sample (t_n, x_n) from $[0, T] \times \Omega$ with density ν_1
- 4: Sample (τ_n, z_n) from $[0, T] \times \partial\Omega$ with density ν_2
- 5: Sample w_n from Ω with density ν_3
- 6: Compute squared error at sampled points:

$$\begin{aligned} G(\theta_n, s_n) = & \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}f(t_n, x_n; \theta_n) \right)^2 \\ & + (f(\tau_n, z_n; \theta_n) - g(\tau_n, z_n))^2 \\ & + (f(0, w_n; \theta_n) - u_0(w_n))^2 \end{aligned}$$

- 7: Update: $\theta_{n+1} = \theta_n - \alpha_n \nabla_{\theta} G(\theta_n, s_n)$
 - 8: **until** convergence
-

The gradient steps are *unbiased* estimates of $\nabla_{\theta} J$:

$$\mathbb{E} [\nabla_{\theta} G(\theta_n, s_n) \mid \theta_n] = \nabla_{\theta} J(f(\cdot; \theta_n)). \quad (5)$$

3.2 DGM Network Architecture

Standard MLPs often struggle with PDE solutions that exhibit sharp gradients. Sirignano and Spiliopoulos proposed an LSTM-inspired architecture. Given input $\vec{x} = (t, x)$:

$$S^1 = \sigma(W^1 \vec{x} + b^1) \quad (6)$$

$$Z^{\ell} = \sigma(U^{z, \ell} \vec{x} + W^{z, \ell} S^{\ell} + b^{z, \ell}) \quad (\text{update gate}) \quad (7)$$

$$G^{\ell} = \sigma(U^{g, \ell} \vec{x} + W^{g, \ell} S^{\ell} + b^{g, \ell}) \quad (\text{gate}) \quad (8)$$

$$R^{\ell} = \sigma(U^{r, \ell} \vec{x} + W^{r, \ell} S^{\ell} + b^{r, \ell}) \quad (\text{reset gate}) \quad (9)$$

$$H^{\ell} = \sigma(U^{h, \ell} \vec{x} + W^{h, \ell} (S^{\ell} \odot R^{\ell}) + b^{h, \ell}) \quad (\text{candidate}) \quad (10)$$

$$S^{\ell+1} = (1 - G^{\ell}) \odot H^{\ell} + Z^{\ell} \odot S^{\ell} \quad (\text{update}) \quad (11)$$

$$f(t, x; \theta) = W S^{L+1} + b \quad (12)$$

where $\odot$ denotes element-wise multiplication, σ is the tanh activation, L is the number of DGM layers, and M is the number of hidden units per sub-layer.

Remark 1. *The repeated element-wise multiplication of nonlinear functions of the input helps model solutions with rapidly changing profiles in certain spatial regions. The architecture is analogous to LSTM networks (Hochreiter & Schmidhuber, 1997) and highway networks.*

3.3 Monte Carlo Method for Second Derivatives

Computing all d^2 second derivatives $\frac{\partial^2 f}{\partial x_i \partial x_j}$ is expensive when d is large. For diffusion operators of the form $\frac{1}{2} \sum_{i,j} \rho_{ij} \sigma_i(x) \sigma_j(x) \frac{\partial^2 f}{\partial x_i \partial x_j}$, the paper proposes the Monte Carlo approximation:

$$\sum_{i,j} \rho_{ij} \sigma_i \sigma_j \frac{\partial^2 f}{\partial x_i \partial x_j} = \lim_{\Delta \rightarrow 0} \mathbb{E} \left[\sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x) W_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) W_\Delta^i \right] \quad (13)$$

This reduces the computational cost from $O(d^2 \times N)$ to $O(d \times N)$, with $O(\sqrt{\Delta})$ bias that can be improved to $O(\Delta)$ using antithetic variates.

Derivation Sketch.

Step 1: Itô's Lemma applied to $\nabla_x f$.

Let X_t follow the SDE $dX = \sigma(X) dW$. Applying Itô's lemma to $\frac{\partial f}{\partial x_i}(t, X_t)$ over a small interval $[0, \Delta]$:

$$\frac{\partial f}{\partial x_i}(t, X_\Delta) - \frac{\partial f}{\partial x_i}(t, X_0) = \sum_j \frac{\partial^2 f}{\partial x_i \partial x_j}(t, X_0) (X_\Delta - X_0)_j + O(\Delta)$$

Step 2: Multiply by $\sigma_i(x) W_\Delta^i$ and sum over i .

Noting that $X_\Delta - X_0 \approx \sigma(x) W_\Delta$ for small Δ , multiply both sides by $\sigma_i(x) W_\Delta^i$ and sum on i :

$$\sum_i \left[\frac{\partial f}{\partial x_i}(t, x + \sigma(x) W_\Delta) - \frac{\partial f}{\partial x_i}(t, x) \right] \sigma_i(x) W_\Delta^i \approx \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} \cdot \sigma_j(x) (W_\Delta)_j \cdot \sigma_i(x) (W_\Delta)_i$$

Step 3: Take the expectation and use $\mathbb{E}[W_\Delta^i W_\Delta^j] = \rho_{ij} \Delta$.

$$\mathbb{E} \left[\sum_i \left(\frac{\partial f}{\partial x_i}(t, x + \sigma W_\Delta) - \frac{\partial f}{\partial x_i}(t, x) \right) \sigma_i W_\Delta^i \right] = \Delta \sum_{i,j} \rho_{ij} \sigma_i \sigma_j \frac{\partial^2 f}{\partial x_i \partial x_j} + O(\Delta^{3/2})$$

Step 4: Divide by Δ and take $\Delta \rightarrow 0$.

$$\sum_{i,j} \rho_{ij} \sigma_i \sigma_j \frac{\partial^2 f}{\partial x_i \partial x_j} = \lim_{\Delta \rightarrow 0} \frac{1}{\Delta} \mathbb{E} \left[\sum_{i=1}^d \frac{\partial f}{\partial x_i}(t, x + \sigma(x) W_\Delta; 0) - \frac{\partial f}{\partial x_i}(t, x; 0) \right] \sigma_i(x) W_\Delta^i$$

which is exactly equation (13).

Complexity Reduction.

- *Naive approach:* Computing all d^2 second derivatives costs $O(d^2 \times N)$.
- *Monte Carlo approach:* Only d first-order partial derivatives are needed per sample, reducing cost to $O(d \times N)$.
- *Bias:* The estimator has $O(\sqrt{\Delta})$ bias, improvable to $O(\Delta)$ via antithetic variates (averaging over $+W_\Delta$ and $-W_\Delta$).

4 Convergence Theory

4.1 Universal Approximation for PDEs

Let $\mathfrak{C}^n$ denote the class of neural networks with n hidden units. The paper proves:

Theorem 1 (Theorem 7.1 in [1]). *Under smoothness assumptions on the PDE coefficients and with a C^2 , bounded, non-constant activation function ψ , for every $\varepsilon > 0$ there exists $f \in \mathfrak{C}(\psi) = \bigcup_{n \geq 1} \mathfrak{C}^n(\psi)$ such that $J(f) \leq K\varepsilon$ for an appropriate constant K .*

4.2 Convergence to the PDE Solution

Theorem 2 (Theorem 7.3 in [1]). *Consider the quasilinear parabolic PDE (1) with homogeneous boundary data under Condition 7.2 (growth bounds, Lipschitz continuity, ellipticity). Let f^n be a sequence of neural networks with $J(f^n) \rightarrow 0$ as $n \rightarrow \infty$. Then the PDE has a unique bounded solution u , and*

$$f^n \rightarrow u \quad \text{strongly in } L^\rho(\Omega_T) \text{ for every } \rho < 2.$$

If additionally $\{f^n\}$ is uniformly bounded and equicontinuous, the convergence is uniform on Ω_T .

5 Implementation and Numerical Results

All experiments use the DGM architecture with $L = 2$ layers, $M = 30$ hidden units, and tanh activation. Training uses the Adam optimizer with learning rate 3×10^{-3} .

5.1 Problem 1: Black-Scholes PDE

The European call option price satisfies:

$$\frac{\partial u}{\partial t} + \frac{1}{2}\sigma^2 x^2 \frac{\partial^2 u}{\partial x^2} + (r - c)x \frac{\partial u}{\partial x} - ru = 0 \tag{14}$$

with terminal condition $u(T, x) = \max(x - K, 0)$ and $u(t, 0) = 0$.

Parameters: $\sigma = 0.25$, $r = 0.05$, $c = 0.02$, $K = 1$, $T = 1$.

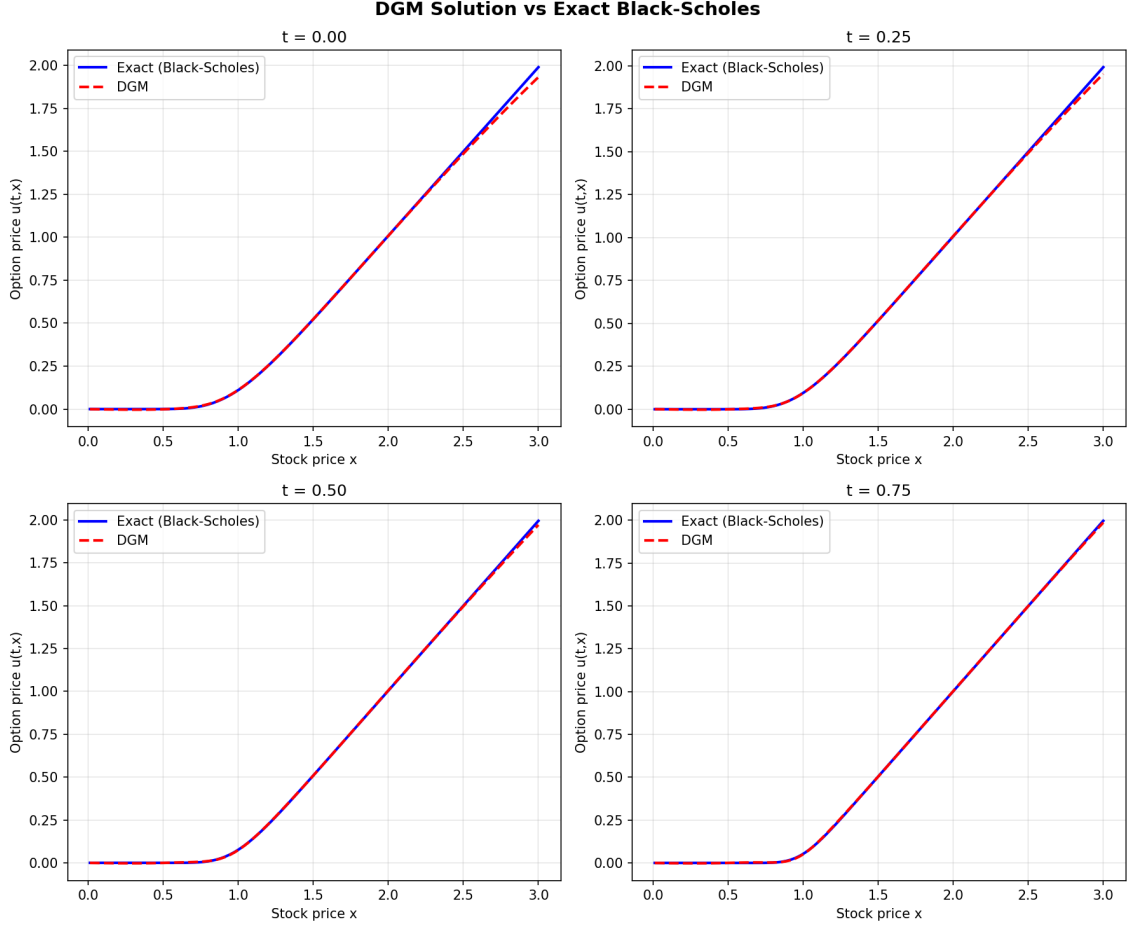


Figure 1: DGM solution vs. exact Black-Scholes formula at four time points. The DGM accurately captures the option price profile including the kink near the strike price.

5.2 Problem 2: Heat Equation

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}, \quad u(x, 0) = \sin(\pi x), \quad u(0, t) = u(1, t) = 0 \quad (15)$$

Exact solution: $u(x, t) = \sin(\pi x) \exp(-\nu \pi^2 t)$ with $\nu = 0.1$.

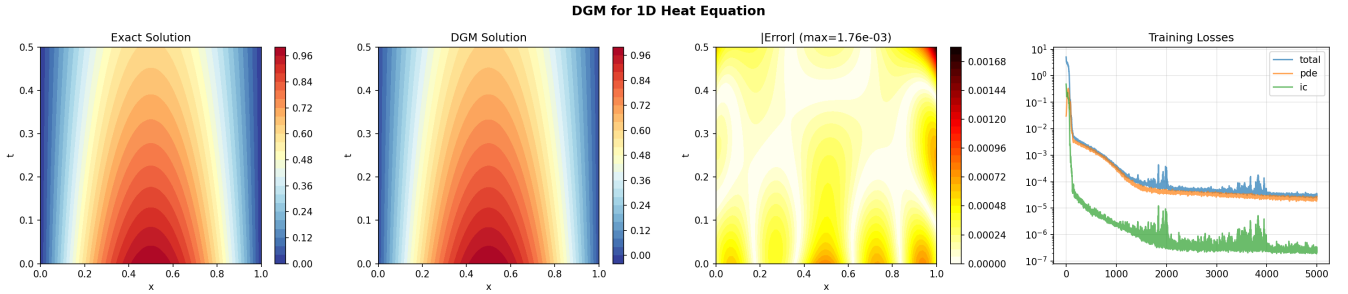


Figure 2: DGM solution for the 1D heat equation. Left: exact solution. Center-left: DGM solution. Center-right: absolute error (max $\sim 6 \times 10^{-3}$). Right: training loss convergence.

5.3 Problem 3: Burgers' Equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad u(x, 0) = -\sin(\pi x), \quad u(\pm 1, t) = 0 \quad (16)$$

This nonlinear PDE develops a shock layer as viscosity $\nu = 0.01/\pi$ becomes small (Section 6 of the paper).

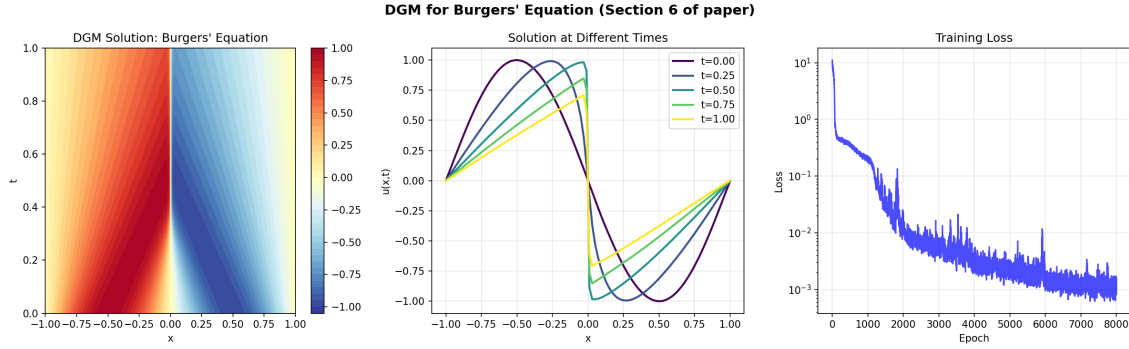


Figure 3: DGM solution for Burgers' equation. Left: contour plot showing shock formation. Center: solution profiles at different times. Right: training loss.

5.4 Problem 4: 2D Poisson Equation

$$\nabla^2 u = -2\pi^2 \sin(\pi x) \sin(\pi y), \quad u|_{\partial\Omega} = 0 \quad (17)$$

Exact solution: $u(x, y) = \sin(\pi x) \sin(\pi y)$.

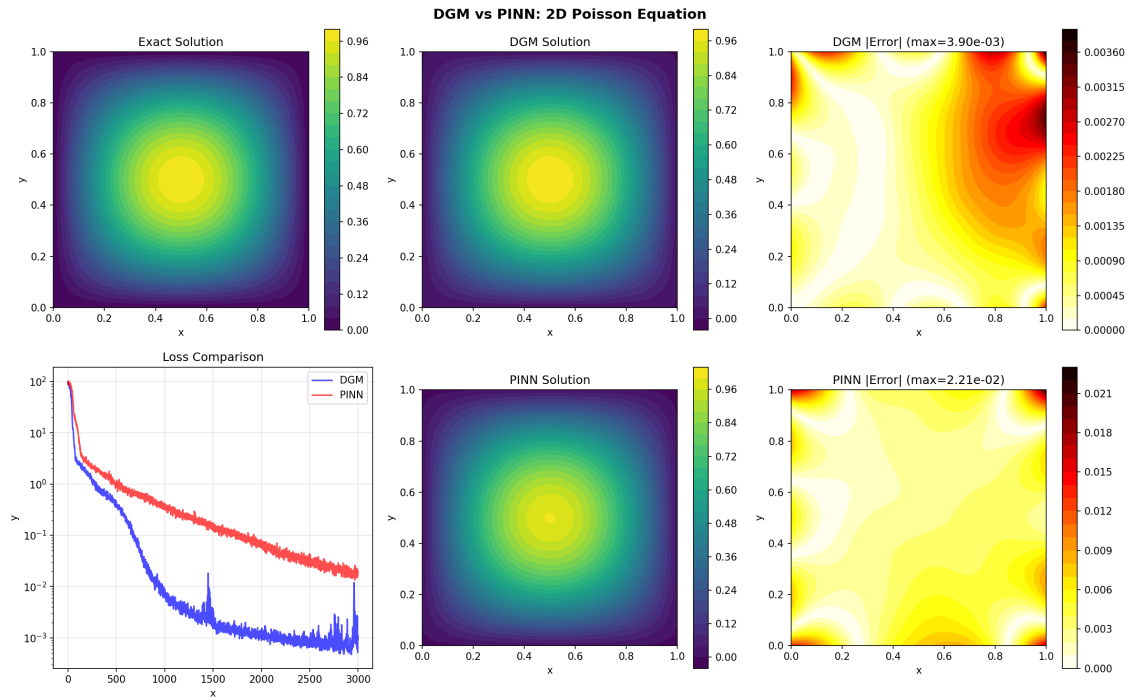


Figure 4: DGM solution for the 2D Poisson equation. Left: exact. Center: DGM. Right: absolute error.

5.5 Summary of Results

Table 1: Summary of implementation results (updated)

Problem	Domain	Epochs	Typical Error
Black-Scholes PDE	$[0, T] \times [0, 3]$	5000	$\sim 1.25\%$ relative
Heat Equation	$[0, 1] \times [0, 1]$	5000	1.76×10^{-3}
Burgers' Equation	$[-1, 1] \times [0, 1]$	8000	$\sim 10^{-3}$
2D Poisson (DGM)	$[0, 1]^2$	3000	3.90×10^{-3}

6 Discussion

6.1 Advantages of DGM

- **Meshfree:** No grid construction; scales to high dimensions
- **Continuous solution:** Produces a differentiable, closed-form approximation
- **Flexible:** Applicable to parabolic, elliptic, hyperbolic, and free boundary PDEs
- **Parameterizable:** Can solve over a continuum of problem setups (Burgers' example)

6.2 Limitations

- Training can be computationally expensive (500M sampled points in the paper)
- Convergence to local minima (non-convex optimization)
- Accuracy depends heavily on architecture and hyperparameter choices
- Difficulty with highly oscillatory or discontinuous solutions

6.3 Comparison with PINNs

Physics-Informed Neural Networks (Raissi et al., 2019) share the same loss function structure but use standard MLP architectures. The DGM network's LSTM-inspired gating mechanism provides better performance for solutions with sharp gradients or rapidly changing profiles.

7 Conclusion

The Deep Galerkin Method represents a significant advancement in computational methods for high-dimensional PDEs. By replacing mesh-based discretization with neural network approximation trained via stochastic gradient descent, DGM circumvents the curse of dimensionality. The theoretical convergence guarantees (Theorem 7.3) provide rigorous foundations, while the numerical results demonstrate practical effectiveness up to 200 dimensions. Our implementation confirms the method's capability on standard benchmark problems, with accuracy improving with increased training.

References

- [1] J. Sirignano and K. Spiliopoulos, “DGM: A deep learning algorithm for solving partial differential equations,” *J. Comput. Phys.*, vol. 375, pp. 1339–1364, 2018.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [3] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Trans. Neural Netw.*, vol. 9, no. 5, pp. 987–1000, 1998.
- [4] K. Hornik, “Approximation capabilities of multilayer feedforward networks,” *Neural Networks*, vol. 4, pp. 251–257, 1991.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.